数据库系统原理第七章：规范化（Normalization）详解

一、为什么要规范化？（课件7.3-7.12页）

1.1 问题的根源：数据冗余与异常

让我们通过一个完整的例子来理解规范化的必要性：

原始表格：worker（员工-部门-经理）

worker_name	branch	manager
John	A	Alice
Mary	A	Alice
Tom	B	David
Jane	B	David
Bob	B	David

问题分析：

1. **冗余（Redundancy）**：

- 部门A的经理Alice出现了2次
- 部门B的经理David出现了3次
- 相同的信息重复存储，浪费存储空间

1. **更新异常（Update Anomaly）**：

   假设我们要将部门B的经理从David改为Debbie：

```
UPDATE worker SET manager = 'Debbie' WHERE branch = 'B';
```

如果这个更新语句执行不完整，或者有程序员忘了更新某一行，就会导致：

```
Tom, B, Debbie
Jane, B, David  ← 不一致！同一个部门有两个经理
Bob, B, Debbie
```

1. **插入异常（Insertion Anomaly）**：

   如果想添加一个新部门C，经理是Eve，但还没有员工：

```
INSERT INTO worker VALUES (NULL, 'C', 'Eve');  -- 不行！worker_name不能为空
```

因为worker_name是主键（或候选码），不能为NULL，所以无法插入。

1. **删除异常（Deletion Anomaly）**：

   如果删除了部门B的所有员工：

```
DELETE FROM worker WHERE branch = 'B';
```

那么部门B的经理信息（David）也会被删除，即使部门B本身还存在。

1.2 根本原因分析

这个问题的根本原因是存在不良的函数依赖：`branch → manager`（部门决定经理），但`branch`不是这个表的超码。

函数依赖 ：

- `worker_name → branch`（每个员工只在一个部门）
- `branch → manager`（每个部门只有一个经理）
- 传递依赖：`worker_name → branch → manager`

二、函数依赖详解（课件7.22-7.75页）

2.1 函数依赖的定义

定义 ：设R是一个关系模式，$\alpha \subseteq R$ 和 $\beta \subseteq R$。函数依赖 $\alpha \rightarrow \beta$ 在R上成立，当且仅当对于R的任意两个元组t_1和t_2，如果$t_1[\alpha] = t_2[\alpha]$，则$t_1[\beta] = t_2[\beta]$。

例子1 ：学生表student(SID, name, college, city)

- 如果每个学院只在一个城市：`college → city`
- 每个学号唯一确定一个学生：`SID → name, college, city`
- 但可能存在同名学生，所以：`name → college`不一定成立

例子2 ：选课表SC(SID, CID, grade, course_name)

- (SID, CID) → grade：学号和课程号一起决定成绩
- CID → course_name：课程号决定课程名称
- 但SID → course_name不成立：一个学生可以选多门课

2.2 函数依赖的闭包F^+

给定函数依赖集F，F的闭包F^+是F逻辑蕴含的所有函数依赖的集合。

Armstrong公理 ：

1. **自反律** ：如果$\beta \subseteq \alpha$，则$\alpha \rightarrow \beta$
2. **增广律** ：如果$\alpha \rightarrow \beta$，则$\alpha\gamma \rightarrow \beta\gamma$
3. **传递律** ：如果$\alpha \rightarrow \beta$ 且 $\beta \rightarrow \gamma$，则$\alpha \rightarrow \gamma$

计算闭包的例子 ：

设R(A, B, C)，F = {A → B, B → C}

计算F^+：

1. 自反得到：A → A, B → B, C → C, AB → A, AB → B, ...
2. 增广得到：A → AB, A → AC, B → BC, ...
3. 传递得到：A → C（因为A → B 且 B → C）
4. 继续应用规则，得到所有可能的函数依赖

实际计算方法 ：计算属性集α的闭包α^+

算法：

```
result = α
while result有变化 do
    for each 函数依赖β → γ in F do
        if β ⊆ result then
            result = result ∪ γ
```

例子 ：R(A, B, C, D, E)，F = {AB → C, C → D, D → E}

计算AB的闭包$(AB)^+$：

1. 初始：result = {A, B}

2. AB → C：AB ⊆ result，添加C，result = {A, B, C}
3. C → D：C ⊆ result，添加D，result = {A, B, C, D}
4. D → E：D ⊆ result，添加E，result = {A, B, C, D, E}
5. 最终：$(AB)^+$ = {A, B, C, D, E}

2.3 规范覆盖（Canonical Cover）

规范覆盖Fc是F的一个最小覆盖，满足：

1. Fc中的每个函数依赖右边只有一个属性
2. 删除Fc中的任何函数依赖都会改变Fc的闭包
3. 删除任何函数依赖左边的任何属性都会改变Fc的闭包

计算规范覆盖的例子 ：

F = {A → BC, B → C, A → B, AB → C}

步骤：

1. 分解右边为单个属性：{A → B, A → C, B → C, AB → C}

2. 检查A → C是否冗余：计算A^+ = {A, B, C}，包含C，所以A → C冗余，删除

3. 检查AB → C是否冗余：

   - 检查A是否冗余：计算$(AB - A)^+ = B^+$ = {B, C}，包含C，所以A冗余
   - AB → C变成B → C，但B → C已存在
   - 所以AB → C完全冗余，删除

4. 最终Fc = {A → B, B → C}

三、范式详解（课件7.27-7.48页）

3.1 第一范式（1NF）

要求 ：每个属性都是原子的，不可再分。

违反1NF的例子 ：

1. **复合属性** ：

```
学生表：Student(SID, name(first, last), age)
应改为：Student(SID, first_name, last_name, age)
```

1. **多值属性** ：

```
学生表：Student(SID, name, phones{phone1, phone2, ...})
应改为两个表：
Student(SID, name)
StudentPhone(SID, phone)
```

1. **表中有表** ：

```
订单表：Order(order_id, customer, items[(product, quantity), ...])
应改为：
Order(order_id, customer)
OrderItem(order_id, product, quantity)
```

1NF的例子 ：

```
Student(SID, name, gender, age)  -- 所有属性都是原子的
Enrollment(SID, CID, grade)      -- 所有属性都是原子的
```

3.2 第二范式（2NF）

要求 ：在1NF基础上，每个非主属性完全函数依赖于候选码。

违反2NF的例子 ：

```
选课表：SC(SID, name, CID, course_name, grade)
函数依赖：
1. SID → name              -- 部分依赖（name依赖于候选码的一部分SID）
2. CID → course_name       -- 部分依赖（course_name依赖于候选码的一部分CID）
3. (SID, CID) → grade      -- 完全依赖
候选码：(SID, CID)
```

问题 ：

- 同一个学生的姓名在每门课中重复存储
- 同一门课的名称在每个选课学生中重复存储

分解为2NF ：

```
Student(SID, name)                -- 满足2NF
Course(CID, course_name)          -- 满足2NF
Enrollment(SID, CID, grade)       -- 满足2NF
```

3.3 第三范式（3NF）

要求 ：在2NF基础上，每个非主属性不传递依赖于候选码。

违反3NF的例子 ：

```
学生表：Student(SID, name, dept_id, dept_name, dept_dean)
函数依赖：
1. SID → name, dept_id
2. dept_id → dept_name, dept_dean
候选码：SID
```

传递依赖 ：SID → dept_id → dept_name

- dept_name不直接依赖于SID，而是通过dept_id传递依赖

问题 ：

- 同一个系的信息在每个学生中重复存储
- 更新系主任时，需要修改该系所有学生的记录

分解为3NF ：

```
Student(SID, name, dept_id)           -- 满足3NF
Department(dept_id, dept_name, dept_dean)  -- 满足3NF
```

3.4 BC范式（BCNF）

要求 ：对于每个非平凡函数依赖$\alpha \rightarrow \beta$，α必须是超码。

BCNF vs 3NF ：

- 3NF允许：β是主属性，或者α是超码
- BCNF要求：α必须是超码

违反BCNF但满足3NF的例子 ：

```
教学表：Teaching(student, teacher, course)
假设：
1. 每位老师只教一门课：teacher → course
2. 每门课可以由多位老师教
3. 每个学生选一门课，但可以有多个老师
函数依赖：
1. teacher → course
```

2. (student, course) → teacher
候选码：(student, teacher) 和 (student, course)

检查BCNF ：

- teacher → course：teacher不是超码（teacher⁺ = {teacher, course}，不包含student）
- 违反BCNF！

分解为BCNF ：

1. 用违反的依赖teacher → course分解：

   R1(teacher, course) -- teacher是候选码，满足BCNF

   R2(student, teacher) -- (student, teacher)是候选码，没有非平凡依赖，满足BCNF

2. 检查依赖保持：

   ◦ teacher → course 保持在R1中

   ◦ (student, course) → teacher 丢失了！

   需要连接R1和R2才能检查(student, course) → teacher

四、分解算法详解（课件7.52-7.112页）

4.1 无损分解测试

定理 ：将R分解为R1和R2是无损的，当且仅当以下至少一个在F^+中：

1. R1 ∩ R2 → R1
2. R1 ∩ R2 → R2

例子 ：

R(A, B, C)，F = {A → B, B → C}

分解为：R1(A, B)，R2(B, C)

计算R1 ∩ R2 = {B}

检查：B → B, C 在F^+中吗？

- B → C 在F中
- 所以B → B, C 在F^+中

  因此分解是无损的。

有损分解的例子 ：

R(A, B, C)，F = {A → B}

分解为：R1(A, B)，R2(A, C)

R1 ∩ R2 = {A}

检查：A → A, B 在F^+中吗？

- A → B 在F中
- 所以A → A, B 在F^+中

看起来是无损的？等等，这是无损的。

有损的例子：R(A, B, C)，F = {AB → C, C → B}

分解为：R1(A, C)，R2(B, C)

R1 ∩ R2 = {C}

检查：C → A, C 吗？即C → A？

C → A不在F^+中

检查：C → B, C 吗？即C → B？

C → B在F中，但C → C是平凡的

所以C → B, C 在F^+中

因此这个分解也是无损的。

真正有损的例子：R(A, B, C)，无函数依赖

分解为：R1(A, B)，R2(B, C)

原始数据：{(a, b, c), (a', b, c')}

投影到R1：{(a, b), (a', b)}

投影到R2：{(b, c), (b, c')}

自然连接结果：{(a, b, c), (a, b, c'), (a', b, c), (a', b, c')}

多了两个元组，所以是有损的。

4.2 BCNF分解算法

算法 ：

输入：关系模式R，函数依赖集F
输出：BCNF分解的结果

```
result = {R}
done = false
while not done do
    if result中有模式R_i不属于BCNF
        找到非平凡函数依赖α → β，其中α不是R_i的超码
        将R_i分解为：
        R_i1 = α ∪ β
        R_i2 = R_i - (β - α)
        用R_i1和R_i2替换result中的R_i
    else
        done = true
```

例子 ：R(A, B, C, D)，F = {A → B, B → C}

候选码：AD

步骤：

1. 检查R：A → B违反BCNF（A^+ = ABC，不是超码）
2. 分解：R1 = A ∪ B = (A, B)，R2 = R - (B - A) = (A, C, D)
3. 检查R1(A, B)：A → B，A是超码，满足BCNF

4. 检查R2(A, C, D)：函数依赖A → C（由A → B和B → C传递得到）

   A^+ = A, C，不是超码（超码需要能推出所有属性）

5. 分解R2：R21 = A ∪ C = (A, C)，R22 = R2 - (C - A) = (A, D)

6. 最终：R1(A, B)，R21(A, C)，R22(A, D) 都满足BCNF

4.3 3NF分解算法

算法 ：

1. 计算F的规范覆盖Fc
2. 为Fc中每个函数依赖α → β创建关系模式(α ∪ β)
3. 如果没有任何关系模式包含R的候选码，添加一个只包含候选码的关系模式
4. 删除被包含的关系模式

例子 ：R(A, B, C, D, E)，F = {A → B, BC → D, D → E}

候选码：AC

步骤：

1. 计算规范覆盖Fc（假设已经是规范覆盖）

2. 创建关系：

   R1(A, B) -- 来自A → B

   R2(B, C, D) -- 来自BC → D

   R3(D, E) -- 来自D → E

3. 添加候选码关系：R4(A, C)

4. 检查包含关系：无重复

5. 最终：{R1, R2, R3, R4}

五、多值依赖和4NF（课件7.115-7.126页）

5.1 多值依赖的定义

定义 ：设R是关系模式，α、β是属性子集，γ = R - αβ。多值依赖α →→ β成立，当且仅当：

对于任意两个元组t1, t2 ∈ R，如果t1[α] = t2[α]，则存在两个元组t3, t4 ∈ R，使得：

1. t3[α] = t4[α] = t1[α] = t2[α]
2. t3[β] = t1[β] 且 t3[γ] = t2[γ]
3. t4[β] = t2[β] 且 t4[γ] = t1[γ]

直观理解 ：α →→ β表示α和β独立于γ

例子 ：教学关系Teaching(student, teacher, course)

假设：

- 每个学生可以有多个老师
- 每个老师可以教多门课
- 每个学生可以选择多门课

- 但老师和他教的课是独立的

数据可能如下：

```
student  teacher  course
John     Smith    Math
John     Smith    Physics
John     Jones    Math
John     Jones    Physics
```

这里：student →→ teacher 且 student →→ course

5.2 第四范式（4NF）

定义 ：关系模式R属于4NF，当且仅当对于每个非平凡的多值依赖α →→ β，α是R的超码。

违反4NF的例子 ：

R(ID, child_name, phone_number)

假设：

- 每个ID有多个孩子
- 每个ID有多个电话号码
- 孩子和电话号码独立

多值依赖：

- ID →→ child_name
- ID →→ phone_number

但ID不是超码（需要child_name和phone_number一起才能唯一标识元组）

分解为4NF ：

R1(ID, child_name)

R2(ID, phone_number)

六、真题解析（期末考试试卷相关题目）

6.1 填空题第6题

"在关系数据库的规范化中，既具有无损连接性，又保持函数依赖的分解，最高能分解到_____范式。"

答案 ：3NF

解释 ：BCNF分解可能无法保持函数依赖，而3NF分解算法可以保证既无损又保持依赖。

6.2 综合题第2题

题目：给定关系模式R(商店编号S, 商品编号I, 数量N, 部门编号D, 负责人M)，函数依赖集F = {(S,I)→D, (S,D)→M, (S,I)→N}。

解题步骤 ：

1. **找出所有候选码**
    - 计算$(S,I)^+$：
        - 初始：{S, I}
        - 由(S,I)→D：添加D，{S, I, D}

    - 由(S,D)→M：添加M，{S, I, D, M}
    - 由(S,I)→N：添加N，{S, I, D, M, N}
    - 所以(S,I)是候选码
  - 检查其他属性组合：S^+、I^+、$(S,D)^+$等都不能推出所有属性
  - 所以候选码只有：(S, I)

2. **判断R最高属于第几范式**
  - 检查2NF：非主属性{N, D, M}是否完全依赖于候选码(S,I)？
    - (S,I)→N：完全依赖
    - (S,I)→D：完全依赖
    - (S,I)→M：传递依赖？计算：由(S,I)→D和(S,D)→M，所以(S,I)→M是传递依赖
    - 但传递依赖是3NF的问题，2NF只关心部分依赖
    - 没有非主属性依赖于候选码的一部分，所以满足2NF
  - 检查3NF：是否存在非主属性传递依赖于候选码？
    - M通过D传递依赖于(S,I)：满足3NF条件3（β是主属性）？D不是主属性
    - 所以存在非主属性M传递依赖于候选码，违反3NF
  - 结论：R最高属于2NF

3. **分解为BCNF**
  - 找到违反BCNF的依赖：(S,D)→M
    - $(S,D)^+$ = {S, D, M}，不是超码
  - 分解：

    R1(S, D, M)

    R2(S, I, N, D)
  - 检查R2：
    - 候选码：(S, I)
    - 函数依赖：(S,I)→N, (S,I)→D
    - 没有违反BCNF的依赖
  - 最终分解：R1(S, D, M) 和 R2(S, I, N, D)

4. **分解为3NF并保持无损连接和函数依赖**
  - 计算规范覆盖Fc：

    F = {(S,I)→D, (S,D)→M, (S,I)→N}

    已经是规范覆盖（右边单属性，无冗余依赖，左边无冗余属性）
  - 为每个依赖创建关系：

    R1(S, I, D) -- 来自(S,I)→D

    R2(S, D, M) -- 来自(S,D)→M

    R3(S, I, N) -- 来自(S,I)→N

- 检查候选码(S,I)是否在某个关系中：在R1和R3中
- 检查是否有关系被包含：R3(S,I,N)被R1(S,I,D)包含？不，N不在R1中
- 但我们可以合并R1和R3：R13(S, I, D, N)
- 最终分解：R13(S, I, D, N) 和 R2(S, D, M)
- 检查无损连接：R13∩R2 = {S, D}，且(S,D)→(S,D,M)成立
- 检查依赖保持：
  - (S,I)→D 在R13中
  - (S,D)→M 在R2中
  - (S,I)→N 在R13中

    所有依赖都保持

七、总结与学习建议

7.1 范式总结

范式	要求	解决的问题	判断方法
1NF	属性原子	重复组、嵌套表	检查是否有复合或多值属性
2NF	完全依赖	部分依赖导致的冗余	非主属性是否依赖于候选码的一部分
3NF	无传递依赖	传递依赖导致的冗余	非主属性是否直接依赖于候选码
BCNF	决定因素是超码	所有非平凡依赖左边是超码	检查每个α→β，α是否是超码
4NF	无多值依赖	多值依赖导致的冗余	检查每个α→→β，α是否是超码

7.2 分解原则

1. **无损分解** ：自然连接能恢复原关系
2. **依赖保持** ：分解后所有函数依赖可验证
3. **BCNF** ：可能无法保持依赖
4. **3NF** ：总能保持依赖且无损

7.3 学习建议

1. **理解概念** ：不要死记硬背，理解每种异常的实际例子
2. **掌握算法** ：熟练计算闭包、规范覆盖、范式判断、分解算法
3. **多做练习** ：通过真题和课件例子巩固
4. **注意特殊情况** ：如全码关系、多值依赖等

7.4 常见易错点

1. **混淆部分依赖和传递依赖** ：

- 部分依赖：非主属性依赖于候选码的一部分
- 传递依赖：非主属性通过其他非主属性间接依赖于候选码

1. **错误判断候选码** ：

- 必须能推出所有属性
- 且是最小的（去掉任何属性就不能推出所有属性）

1. **分解时丢失依赖** ：

- BCNF分解可能丢失依赖
- 3NF分解能保持依赖

1. **忽略多值依赖** ：

- 多值依赖不是函数依赖
- 4NF针对多值依赖

通过系统学习本章内容，结合课件中的例子和真题练习，您将能够掌握规范化的核心概念和方法，为数据库设计打下坚实基础。